

IQR — Intelligent Quality Review

Agentic AI validation of SOX 404 controls — a citation for every claim, a reproducible verdict for every run.

DESIGN DOCUMENT

Version 1.0 · August 2026 · Controls & Compliance

Repository: <https://github.com/git-bonda108/iqr-sox> (private)

Brand: HP Electric Blue / black / white per HP Brand Central; HP Forma DJR substituted with Helvetica/Arial.

1. Executive summary

IQR validates a SOX 404 control end to end: the performer's GRC evidence package goes in; an audit-ready, fully cited validation pack comes out. The engine is built once and configured per control — a compiled, SME-approved Validation Plan decides what runs; a fixed LangGraph topology decides how it runs; tool-using agents supply judgment where evidence is messy; deterministic Python supplies every number.

Three invariants govern every design choice, each enforced by an automated test:

- **Numbers are computed by deterministic Python, never by a model** (a test fails if a model is invoked on any numeric path).
- **Every claim carries a citation that mechanically resolves against hashed evidence** (the citation gate rejects anything else).
- **Same input + same plan version $\Rightarrow$ identical verdict** (asserted across repeated runs).

Status: v1 is implemented, green on 36 automated tests, and passing all five evaluation gates (100% defect recall, 0% false exceptions, 100% citation validity, 100% abstention correctness, 100% reproducibility) against three golden fixture controls with seeded defects. v1 ships in Shadow mode: it runs beside the human reviewer; nothing auto-submits.

2. The problem

One control is a tree of nested evidence. The 404 process document branches into Excel workpapers (multi-tab, live formulas), approval emails whose reply chains carry ZIP attachments holding more workbooks and checklists, screenshots embedded inside workbook tabs, SharePoint links, and reviewer checklists — nesting three levels deep (email $\rightarrow$ ZIP $\rightarrow$ workbook $\rightarrow$ screenshot), nine or more artifacts per control, and four check modalities: recompute the numbers, read the screenshots, order the timestamps, verify the sign-offs.

Today a reviewer validates this by hand. It costs hours per control every period, scales only with headcount, produces tickmarks rather than citations, and fatigue drives the costliest failure: missed defects.

- **RPA moves without understanding** — a recorded script cannot judge an approval email or notice missing evidence; every layout change is an engineering ticket.
- **A lone model understands but loses the thread** — one prompt cannot coherently hold 9+ artifacts; it hallucinates arithmetic, self-approves its own findings, and two runs give two answers.

3. Architecture

The decided architecture is a **compiled-plan, deterministic-runtime LangGraph** with tool-using agents inside nodes and a hard judgment/arithmetic split. Planning — the one genuinely non-deterministic activity — happens once, at design time, under human review. Runtime only executes.

IQR platform - three layers

Documents enter once, deterministically. Agents reason over an addressable evidence graph, grounded by retrieval.

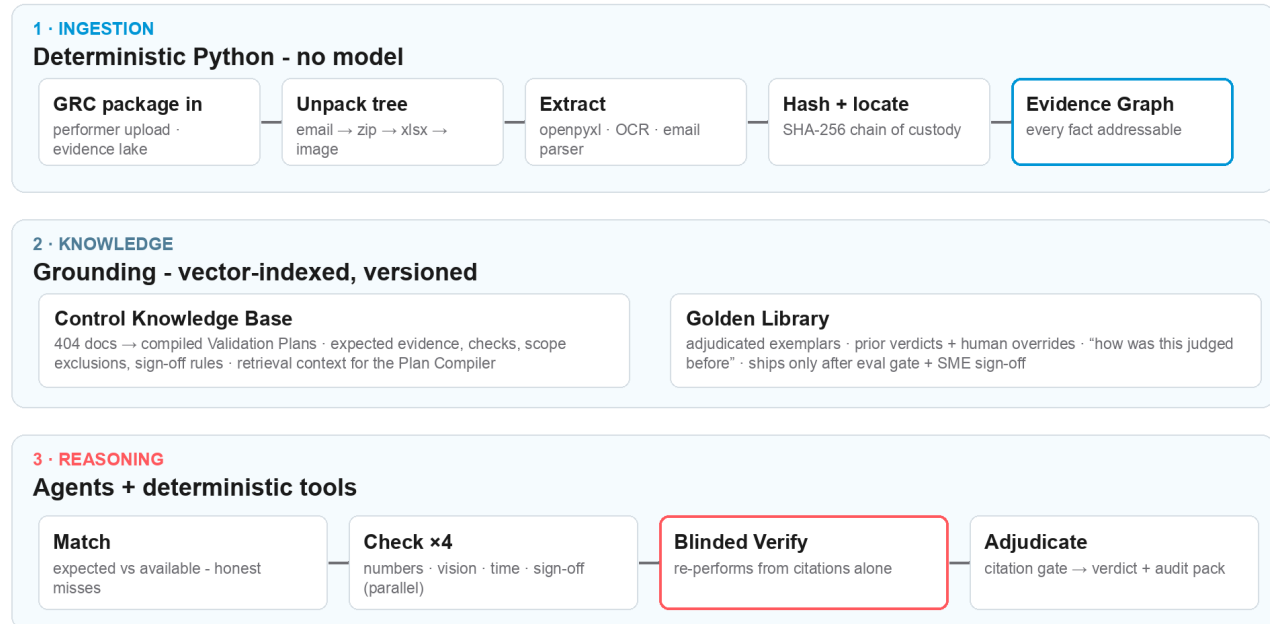


Figure 1 — Three layers: deterministic ingestion, versioned knowledge, agentic reasoning.

3.1 Layer responsibilities

Layer	Responsibility	Key property
Ingestion	Unpack the nested tree, extract cells/text/images/emails, SHA-256 every leaf, build the addressable Evidence Graph	Pure Python — zero model calls; content-addressed custody
Knowledge	Control KB (404s → compiled plans) and Golden Library (adjudicated exemplars), vector-indexed	Versioned; retrieval-anchored grounding
Reasoning	Match evidence, run four check modalities, blinded verification, adjudication	Agents judge within a check; tools do all extraction and computation

3.2 The lifecycle: design once, run every period, learn governed

Compiled-plan lifecycle

Agents where judgment lives, code where numbers live - written once, approved by humans, executed identically every period.

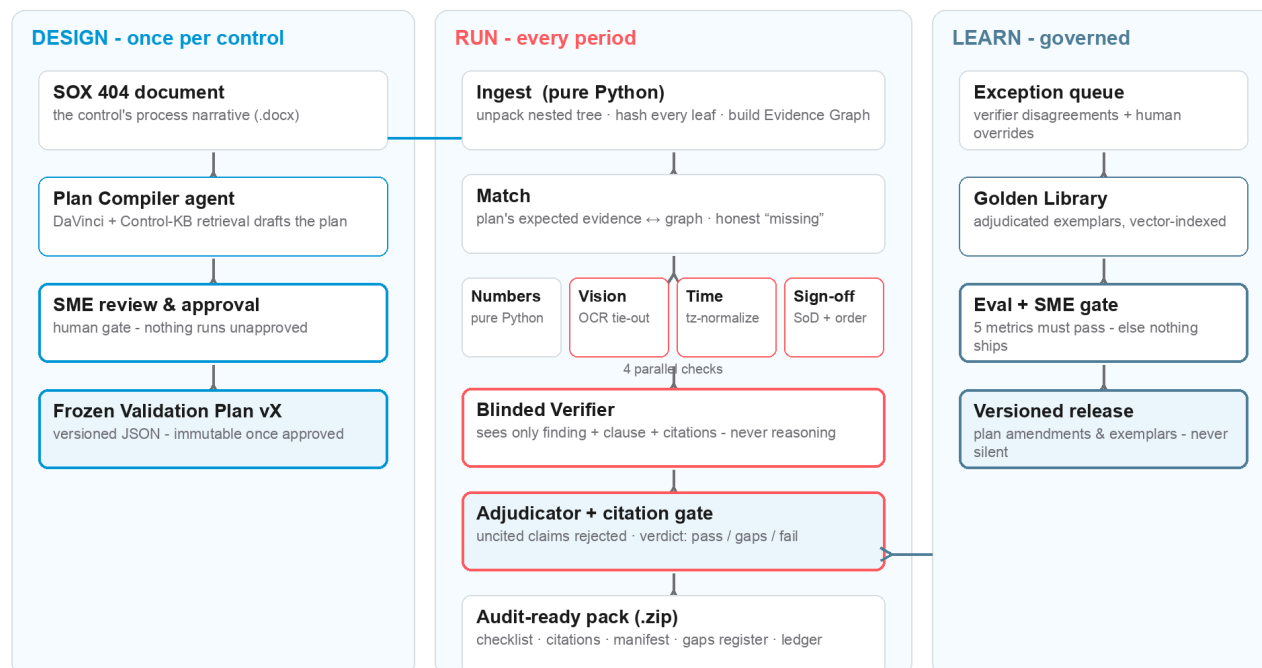


Figure 2 — Compiled-plan lifecycle. The SME approval gate separates judgment about *WHAT* to check from the mechanical *HOW*.

- **Design (once per control):** a Plan Compiler agent reads the 404 document (grounded by Control-KB retrieval) and drafts a Validation Plan — expected evidence with match hints, ordered checks with types and tolerances, scope exclusions, sign-off rules. An SME approves; the plan freezes as versioned, immutable JSON. The runtime refuses unapproved plans.
- **Run (every period):** the fixed graph executes the frozen plan. Agents have judgment within a check but never choose which checks run.
- **Learn (governed):** verifier disagreements and human overrides enter the Golden Library only after the regression eval passes and an SME signs off — versioned, never silent.

4. Agentic orchestration

Run-time topology - fixed, versioned LangGraph

Same shape every run; the topology hash is pinned in every run ledger. Typed state carries the Evidence Graph, never re-dumped text.

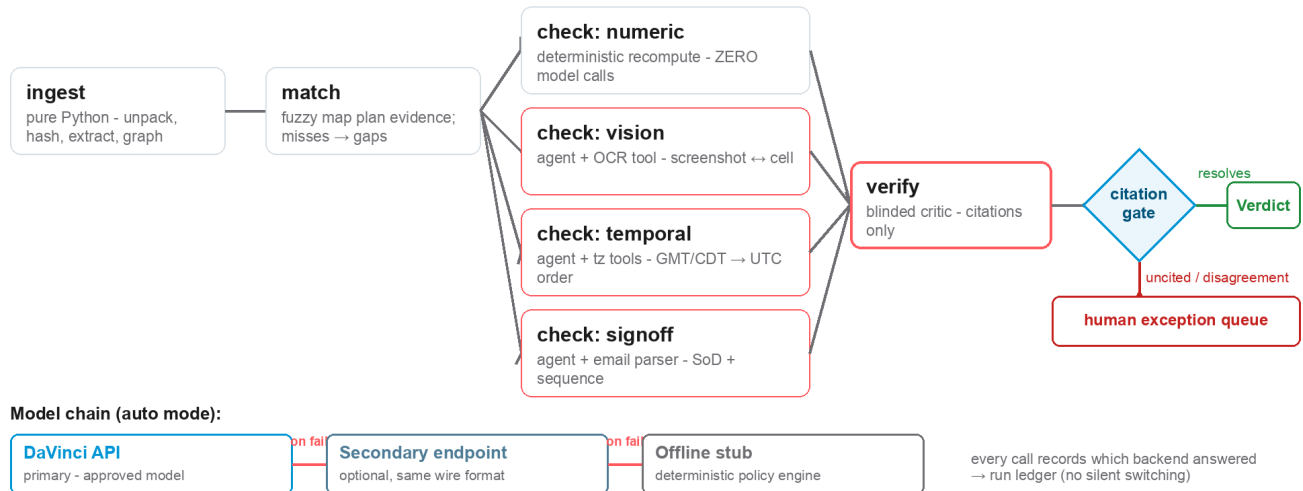


Figure 3 — The fixed LangGraph topology with parallel check fan-out, blinded verification, the citation gate, and the model fallback chain.

4.1 The graph

ingest → match → [one branch per plan check, in parallel] → verify → adjudicate. The topology is serialized and hashed (`topology_signature()`); every run ledger pins the hash, so an auditor can prove the control flow never drifted. State is typed (pydantic models flowing through LangGraph) — the Evidence Graph travels as structure, never re-dumped prompt text, which is what prevents lost-in-the-middle failures.

4.2 Agents and tools — the judgment/arithmetic split

Check type	Executor	Tools used	Model involvement
numeric	Pure Python	cell_read, recompute	NONE — test-enforced
vision	Tool-using agent	ocr_labeled_number (Tesseract), cell_read	judges where the value is and what it means
temporal	Tool-using agent	cell_read, email facts, timestamp_order	judges which stamps matter; tools compute order
signoff	Tool-using agent	email_parse, cell_read, timestamp tools	judges validity of approval; SoD facts from tools

The agent loop is a strict protocol: the model may only respond with a tool call or a final conclusion; every number, timestamp and quotation in a finding is an echo of a tool result, and the runtime — not the model

— accumulates the citations those tools return.

4.3 Blinded verification

The verifier's input type contains exactly three fields — the plan clause, the claimed verdict, and the citations. The executor's reasoning is structurally absent (enforced by the input schema and a test). The verifier re-performs the check from cited evidence alone. Agreement lets the finding stand; disagreement routes it to the human exception queue and the run cannot report a clean pass.

4.4 Model access and multi-mode fallback

All model calls flow through one factory (iqr/config.py), temperature pinned to 0. In **auto** mode each completion tries DaVinci, then an optional secondary endpoint (same wire format), then the deterministic offline stub. Which backend answered is recorded per call in the run ledger — fallback is visible, never silent. If every backend fails, the run fails loudly.

5. The run, step by step

#	Node	Input → Output	Detail
1	ingest	package folder → EvidenceGraph	Recursive unpack (email → zip → workbook → image); SHA-256 per leaf into a content-addressed immutable store; cells (values + formulas), images, emails extracted; corrupt containers and unreadable workbooks recorded in graph.errors — the leaf stays in custody, its facts are absent
2	match	plan + graph → matches / missing	Fuzzy name/path matching per hint; emails resolve to message-ids, screenshots to image hashes; required-but-absent evidence becomes an honest gap; scope-excluded artifacts are never flagged
3	check ×N	matches → Finding per check	Dispatched by frozen check_type; parallel where independent; a crashed check abstains as a gap and routes to the exception queue — it can never sink the run or fake a pass
4	verify	findings → verified findings	Blinded re-performance per finding; gap findings re-checked against the match table (you cannot re-read absent evidence)
5	adjudicate	findings → Verdict	Mechanical citation gate (uncited ⇒ rejected into exceptions); any fail ⇒ fail; gaps/exceptions ⇒ pass_with_gaps; else pass
6	pack	Verdict → audit .zip	verdict.json, auto-completed checklist.md, artifact_manifest.json (every leaf + hash), gaps_and_observations.md, citations.json, plan.json — assembly re-asserts the citation gate

6. Technical stack

Concern	Technology	Why
Orchestration	LangGraph (local)	fixed, versioned, replayable topology — the audit artifact
Agent runtime	Tool-loop over the approved model (Agents SDK-compatible)	judgment constrained to tool calls + typed conclusions
Model	DaVinci API → secondary → offline stub	one approved model; tools decompose the work
Schemas / state	pydantic v2	typed plans, graph, findings; blindness enforced by types
Spreadsheets	openpyxl	cells, formulas, embedded images
OCR	Tesseract via pytesseract	deterministic screenshot reads with regions
Email	Python stdlib email (+ extract-msg)	headers, body lines, nested attachments
Timestamps	python-dateutil + tz maps	GMT/CDT/etc. normalized to UTC before comparison
API / console	FastAPI + single-page console	thin client; all logic server-side
Storage	content-addressed stores + JSONL ledgers	immutable, inspectable, portable
Vector index	local store behind a VectorStore interface	swap in the approved vendor without touching callers
Tests / eval	pytest — 36 tests + 5-gate harness	the invariants are executable

7. Storage & data design

Deployment & storage

Single-machine v1: clone, install, add keys, run. Every store is a plain directory - inspectable, portable, auditable.

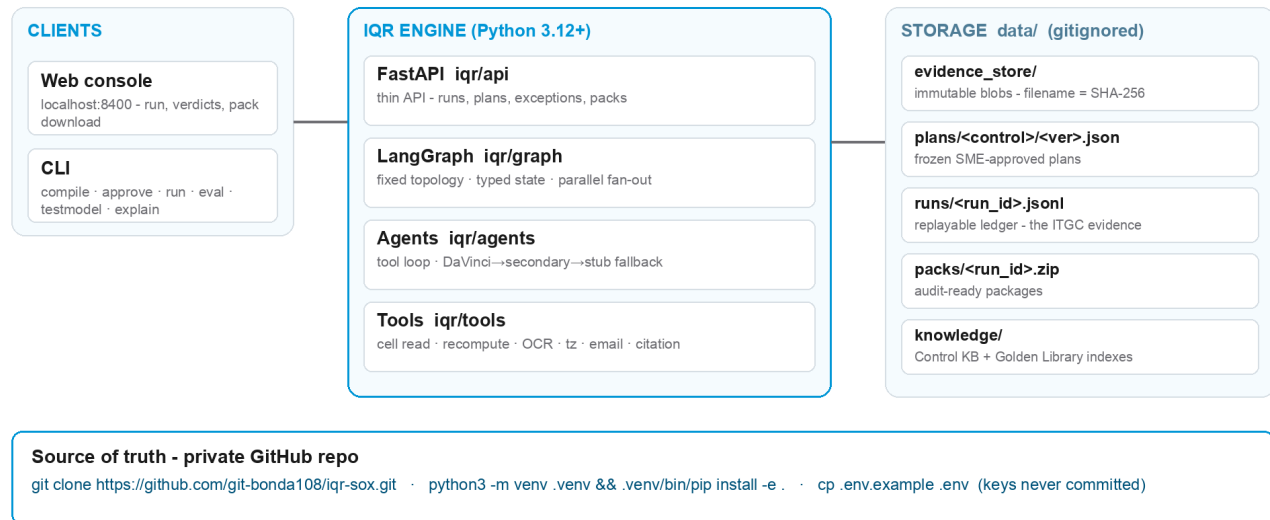


Figure 4 — Deployment and storage. Every store is a plain directory: inspectable, portable, auditable.

- **data/evidence_store/** — blobs named by SHA-256. Same bytes ⇒ same name ⇒ idempotent writes; tampering is self-evident.
- **data/plans/<control>/<version>.json** — frozen plans; re-freezing a version is an error; changes require a new version and re-approval.
- **data/runs/<run_id>.jsonl** — the replayable ledger: every node event, tool call, agent conclusion (with serving backend), verification, adjudication. The platform's own ITGC evidence and its explainability trail.
- **data/packs/<run_id>.zip** — the audit-ready deliverable.
- **data/knowledge/** — Control KB + Golden Library indexes and the pending-overrides journal.

The data/ tree is gitignored: the repository carries code, fixtures and tests; runtime state stays local.

8. API & CLI reference

8.1 HTTP API (FastAPI, localhost:8400)

Method & path	Purpose
POST /api/plans/compile	draft a Validation Plan from a 404 document (doc_path, control_id, frequency)
POST /api/plans/{id}/approve	SME approval — freezes the plan
GET /api/plans/{id}	latest approved plan

Method & path	Purpose
POST /api/runs	execute a control (control_id, package_ref); returns run_id, verdict, pack path
GET /api/runs/{run_id}/ledger	the full replayable ledger
GET /api/runs/{run_id}/pack	download the audit-ready .zip
GET /api/exceptions	pending human adjudications
POST /api/exceptions/adjudicate	record a human adjudication (Golden Library intake)
GET /api/topology	the versioned topology signature (hash)

8.2 CLI

```
python -m iqr.cli compile <404.docx> <control_id> <frequency>
python -m iqr.cli approve <control_id> <sme>
python -m iqr.cli run <control_id> <package_dir>
python -m iqr.cli eval          # five gate metrics
python -m iqr.cli testmodel     # verify API keys / fallback chain
python -m iqr.cli explain <run_id> # replay the ledger (XAI)
python -m iqr.cli serve         # web console on :8400
```

9. Getting started (clone → keys → validate)

```
git clone https://github.com/git-bonda108/iqr-sox.git IQR && cd IQR
python3 -m venv .venv && .venv/bin/pip install -e .
brew install tesseract # macOS (Windows: UB-Mannheim installer; Linux: apt/yum)
cp .env.example .env # add DAVINCI_API_URL + DAVINCI_API_KEY; keep IQR_MODEL=auto
.venv/bin/python -m iqr.cli testmodel
.venv/bin/python tests/fixtures/build_fixtures.py
.venv/bin/python -m pytest tests/ -q
.venv/bin/python -m iqr.cli eval
.venv/bin/python -m iqr.cli serve
```

Security: keys live only in .env, which is gitignored — never commit it. The stub fallback keeps the platform testable with no keys; the ledger shows which backend served every call.

10. Quality: invariants, evaluation, results

10.1 Seven hard invariants (each is a test)

Invariant	Enforced by
No model math on numeric paths	model-call counter asserted unchanged
Citation gate — uncited claims cannot ship	fabricated locators rejected at adjudication AND pack assembly
Reproducibility — identical verdict across runs	verdict fingerprints compared over repeated runs
Honest missing — absent evidence ⇒ gap, never pass	sign-off removal variant
Scope respected — excluded items never flagged	IC-eliminations exclusion vs a stray artifact
Blinded verify — reasoning cannot reach the critic	verifier input schema has no such field; tamper test
Chain of custody — stable hashes in citations	re-ingest hash equality

10.2 Five-gate evaluation harness

Three golden controls (quarterly rebate recompute; monthly consolidation reconciliation with GMT-vs-CDT ordering and email→ZIP→workbook nesting; quarterly EMR review with real-OCR screenshot tie-outs) run clean twice plus five seeded-defect variants: altered total, inverted timestamps, missing sign-off, preparer = reviewer, screenshot mismatch.

Gate metric	Meaning	v1 result
Defect recall	share of planted defects caught (existential)	100%
False-exception rate	clean checks wrongly flagged (reviewer trust)	0%

Gate metric	Meaning	v1 result
Citation validity	claims whose citation resolves (hard gate)	100%
Abstention correctness	absent evidence declared, never invented (honesty)	100%
Reproducibility	identical verdict for the same pack (consistency)	100%

11. Rollout & roadmap

- **Shadow (v1, now):** runs beside the human reviewer; results compared; nothing auto-submits.
- **Assist:** drafts findings; the human decides each one. Gate: measured recall / precision / reproducibility per control.
- **Primary:** the platform decides; humans review exceptions and samples. Per-control gating — each control advances only as fast as its demonstrated accuracy.

Next steps: point the Plan Compiler at the real 404 documents and GRC packages; wire the approved vector-store vendor behind the existing interface; connect mailbox/SharePoint watchers behind the resolver interface; run one full Shadow cycle and let the eval gates decide graduation.